

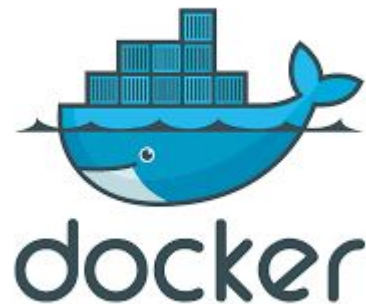
COMS W4156 Advanced Software Engineering (ASE)

December 5, 2023



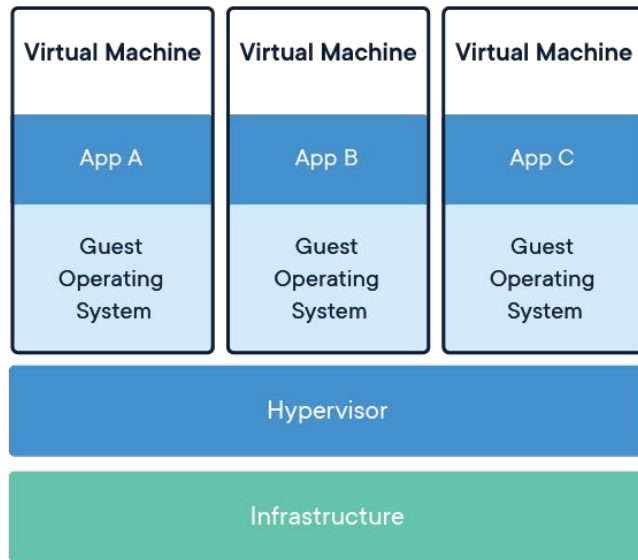
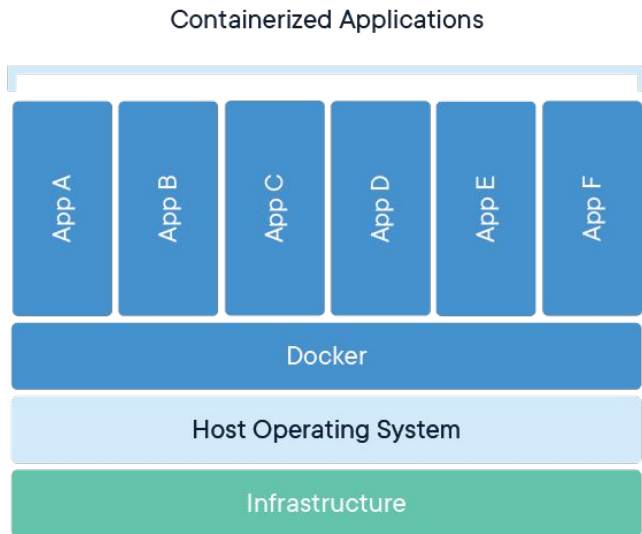
Agenda

1. **docker demo: Xiao Yu**
2. continue design patterns pros and cons
3. test suite quality



Docker Demo: Xiao Yu

- [Google slides](#)
- [Github](#)



Agenda

1. docker demo: Xiao Yu
2. continue design patterns pros and cons
3. test suite quality

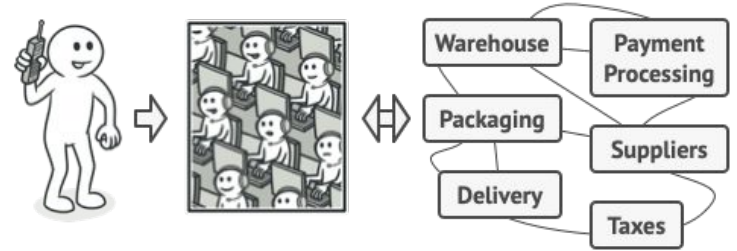


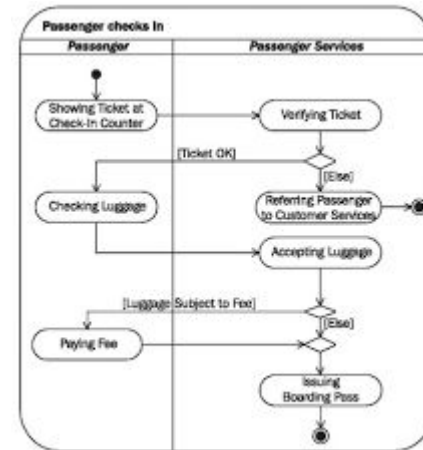
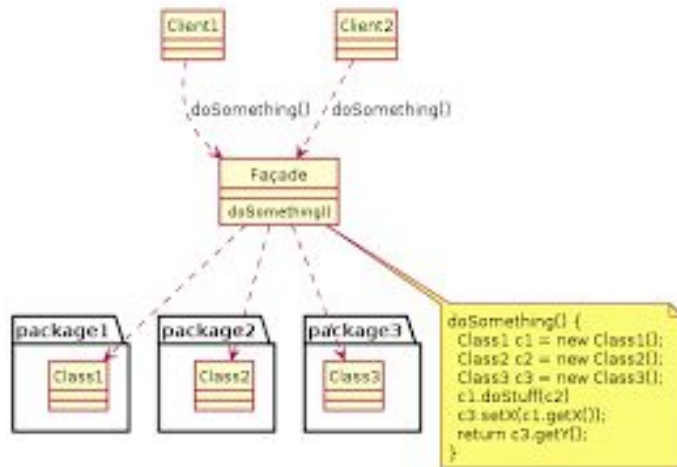
Facade Design Pattern

Facade defines a simplified interface to a subsystem, library, framework, or any other complex set of classes with a lot of moving parts. Provides a shortcut to the most-used features of the subsystem that fit most client requirements

Instead of making your code work with subsystem classes directly, create a facade class that encapsulates that functionality and hides it from the rest of the code. The facade should redirect the calls from the client code to appropriate objects of the subsystem, and be responsible for initializing the subsystem objects and managing their further life cycles

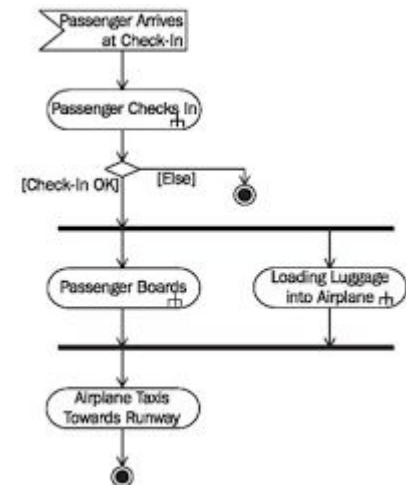
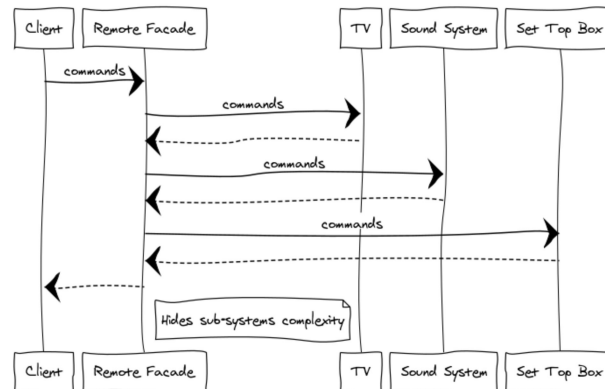
Use when you need a limited but straightforward interface. Also helps to minimize the effort of upgrading to future versions of the subsystem or replacing it with another - the only thing you'd need to change would be the implementation of the facade's methods





Architect: Aseem Jain

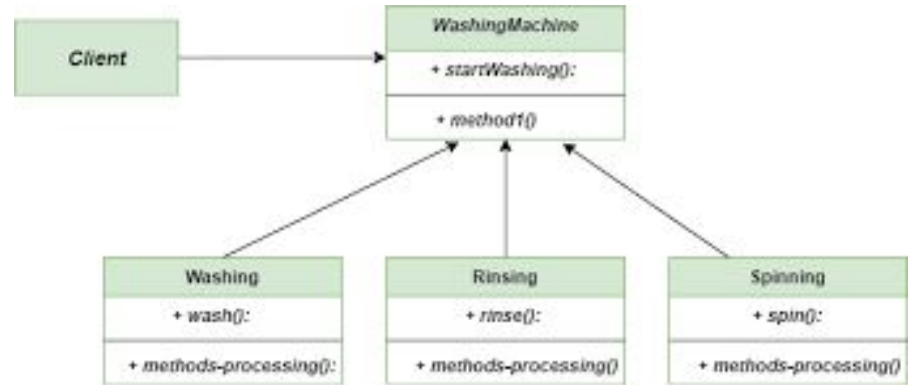
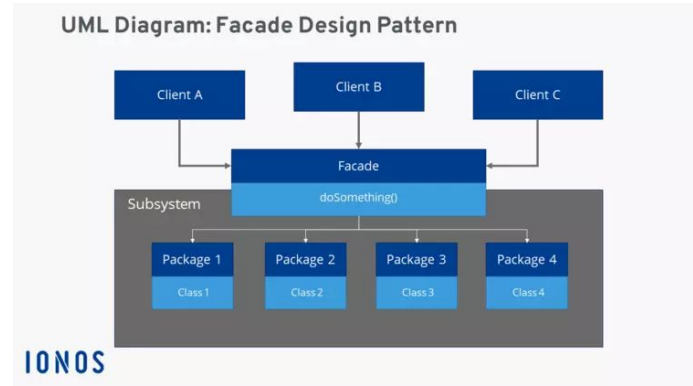
Facade Design Pattern



Facade

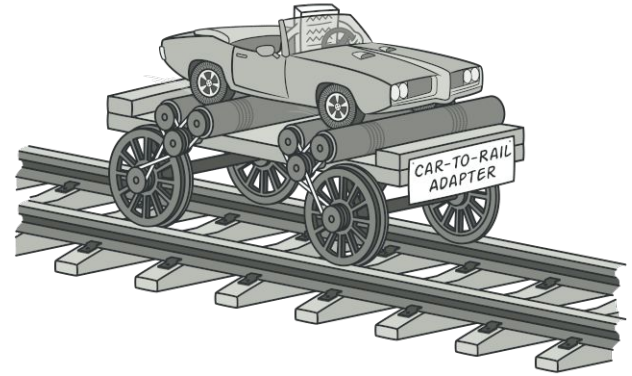
Pros: Reduces complexity, promotes loose coupling, makes it easier to expand the underlying subsystem

Cons: A facade can become a god object coupled to all classes of a system even though it does not introduce any new functionality, additional indirection reduces performance



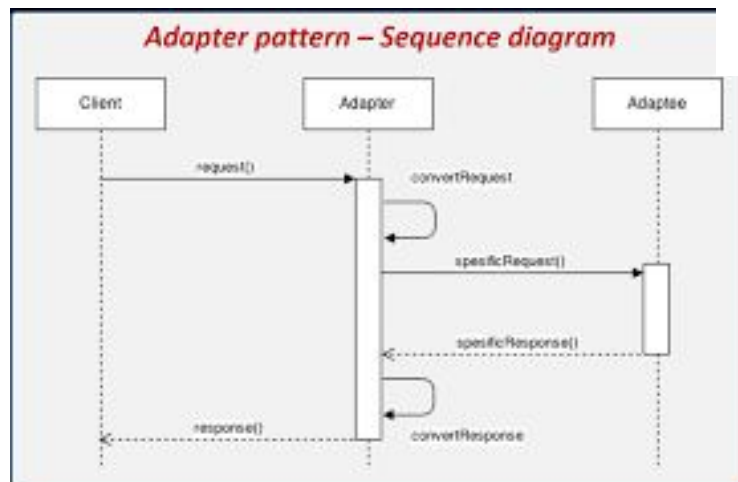
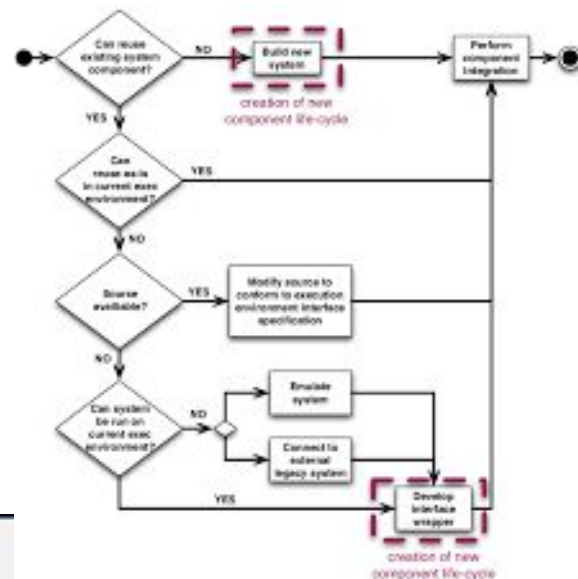
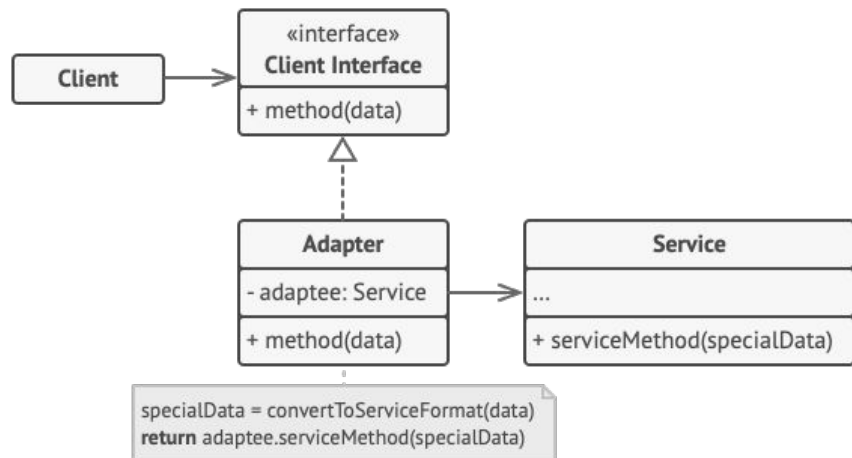
Adapter Design Pattern

[Adapter](#) (or wrapper) allows objects with incompatible interfaces to collaborate by converting the interface of one object so that another object can understand it



The adaptor provides an interface that the client understands and stores a reference to the service object. Upon receiving a call from a client object, the adapter passes the request to the service object in a format and order that it expects, then converts the response(s) to return to the client object

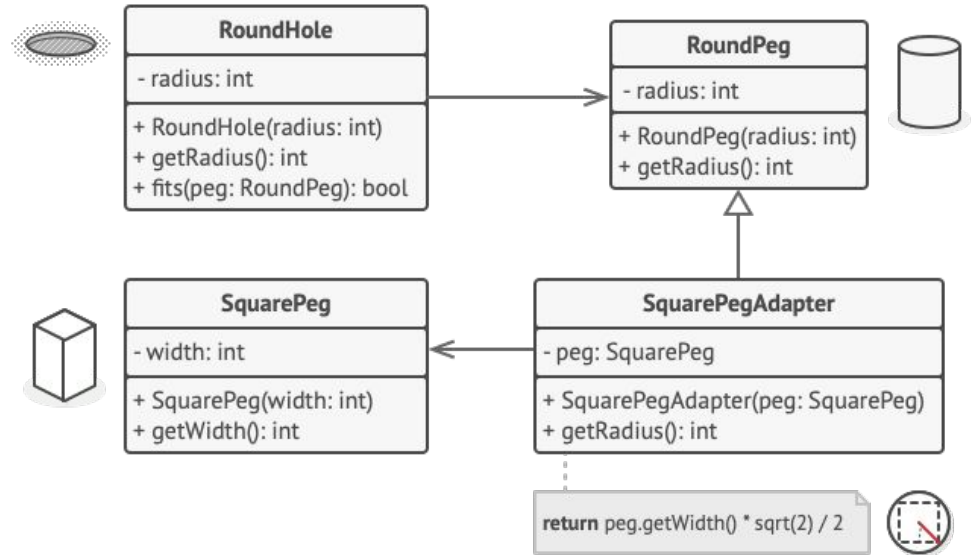
Use when you want to use some existing class, but its interface isn't compatible with the rest of your code - the Adapter pattern creates a middle-layer class to translate between your code and a legacy class, 3rd-party class or any other class that cannot be changed



Adapter

Pros: Data conversion logic is separated from business logic, can replace the service class with a similar alternative by modifying only the adapter not the client

Cons: Introduces overhead, adds another layer of abstraction and indirection to your code that can make it harder to trace the source of errors, identify the root cause of problems, or verify correctness of output



Strategy Design Pattern

Strategy lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable to allow swapping at runtime



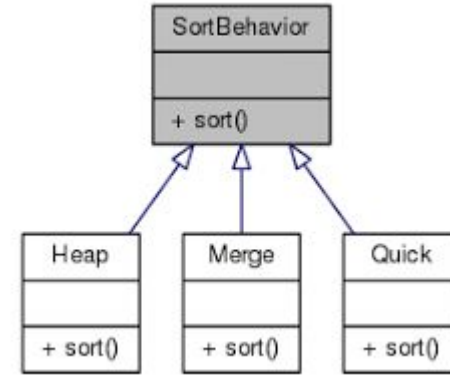
A context class must have a field for storing a reference to one of the strategies. The client passes the desired strategy to the context, which then delegates the work to the linked strategy object through a standard interface common to all strategy objects.

Use when you need different variants of an algorithm and the ability to switch from one to another - the Strategy pattern lets you indirectly alter an object's behavior at runtime by associating it with different objects that perform a specific sub-task in different ways

Strategy

Pros: Replaces complicated switch statements, easily extendable, isolates the implementation details of algorithms from the code that uses them, can replace inheritance with composition

Cons: Clients to the code that uses the strategies must be aware of the differences between strategies to be able to select a proper one



Agenda

1. docker demo: Xiao Yu
2. continue design patterns pros and cons
3. test suite quality



Test Suite Quality

An empty test suite has 0% coverage but all its tests pass!

A non-empty test suite consisting entirely of valid inputs, where all the tests pass, may have 100% coverage but is low quality - ? How could that happen ?

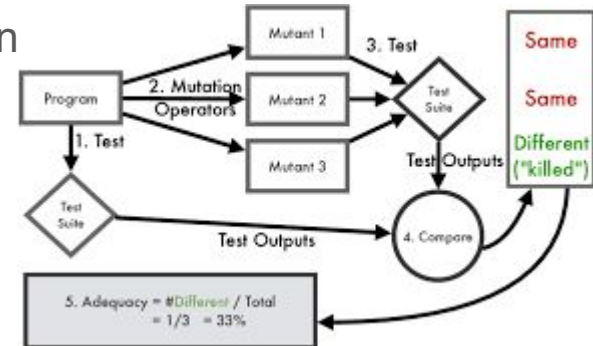
Remember no test suite can cover code that has not been written



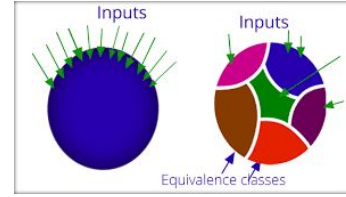
Testing the Test Suite

How do we know whether we have a “good” or “good enough” test suite?

- ✓ [Equivalence Partitioning \(and Boundary Analysis\)](#) - *for outputs as well as inputs*
- ✓ Control Flow Coverage - high percentage of statement, branch, [MC/DC](#) & [MCC](#)
- ✓ [Data Flow Coverage](#) - high percentage of intraprocedural paths from each variable or field definition (write) to everywhere the written value is used (read) before it is overwritten
- ✓ [Mutation Testing](#) - use mutation operators to inject bugs into the code (e.g., switch > to <, insert return statements at arbitrary locations in the middle of functions) and run the regular test suite, did it “kill” (detect) all the mutants?



Equivalence Partitioning for *Inputs*



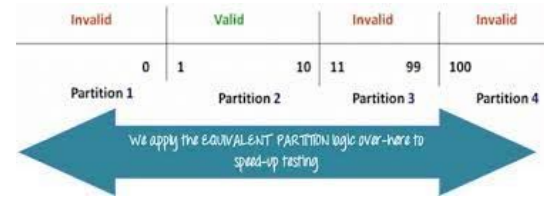
Inputs include any parameters that are read, any global in-memory state that is read, any environment data that is read (from files, databases, network, devices, environment variables, etc.), *and the return values and other shared state received from API, library and system calls that are read* (used for anything) by the code under test. ? What about inputs that are never used ?

Equivalence partitioning of parameters is not enough, also need to test input equivalence partitions for global variables, environment data, and outputs from any external calls that are, effectively, inputs to the code under test. Forcing some of these equivalence partitions may require mocks. ? Why ?

If any of these input values is a status code, need to test with all possible status codes from that source. ? Why does this matter ?

Have both valid and invalid input partitions been tested? Valid vs. invalid is wrt *your* code, not necessarily the code it comes from. ? What's the difference ?

Equivalence Partitioning for *Outputs*



Outputs include any (object, by-reference or data structure) parameters that are written, any global in-memory state that is written, any environment data that is written (files, databases, network, devices, environment variables, etc.), *and the parameters and shared state sent to API, library and system calls* by the code under test. ? Why be concerned with covering output partitions ?

Equivalence partitioning of return values is not enough, also need to test output equivalence partitions for updated parameters, global variables, environment data, and inputs to any external calls. Forcing output equivalence partitions typically requires specially crafted inputs that lead to generating those outputs. ? How should these special inputs be devised ?

If any of these output values is a status code, need tests that produce all possible status codes. ? Why ?

If it's possible for the code under test to produce invalid outputs, then invalid output equivalence partitions need to be covered in addition to valid output partitions. ? When might this happen ?

For many of these cases, we may not know what code might eventually receive these outputs.

? Why does it matter ?

Control Flow Coverage

Multiple Condition Coverage (MCC) vs. Modified Condition/Decision Coverage (MC/DC) rarely used outside safety-critical software

MCDC: each individual condition is true at least once and false at least once [linear]

MCC: conditions must evaluate to T or F in all combinations [combinatoric]

Short circuit: Only need to test conditions that independently affect decision's outcome

(false AND anything-else)

```
if (A and B)
  { do something }
else
  { do something else }
```

```
if (A or B)
  { do something }
else
  { do something else }
```

Example

```
boolean purchaseAlcohol  
(int buyerAge, int ageFriend)  
{  
    boolean allow = False;  
    if ((buyerAge>=21) or  
        (ageFriend>=21))  
    { allow = True; }  
    return allow;  
}
```

Assert purchaseAlcohol(25,25) == True
gives 100% *statement* coverage

Assert purchaseAlcohol(25,25) == True
Assert purchaseAlcohol(20,20) == False
gives 100% *branch* coverage

But *both* the buyer *and* the friend must be
21 or older

Does MCDC coverage detect the bug?

Example continued

```
boolean purchaseAlcohol
(int buyerAge, int ageFriend)
{
    boolean allow = False;
    if ((buyerAge >= 21) or
        (ageFriend >= 21))
    { allow = True; }
    return allow;
}
```

purchaseAlcohol(25,25) == True covers T, T
purchaseAlcohol(20,20) == False covers F, F
purchaseAlcohol(25,20) == False covers T, F
purchaseAlcohol(20,25) == False covers F, T

Does a set of test cases fulfilling MCC
coverage (truth table) detect the bug?

P	Q	$\neg P$	$P \rightarrow Q$	$\neg P \wedge (P \rightarrow Q)$
T	T	F	T	F
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

□

Data Flow Coverage

`x := 12345 ;`

lots of code that does not change x

`y := derive some value from x ;`

some code that changes x

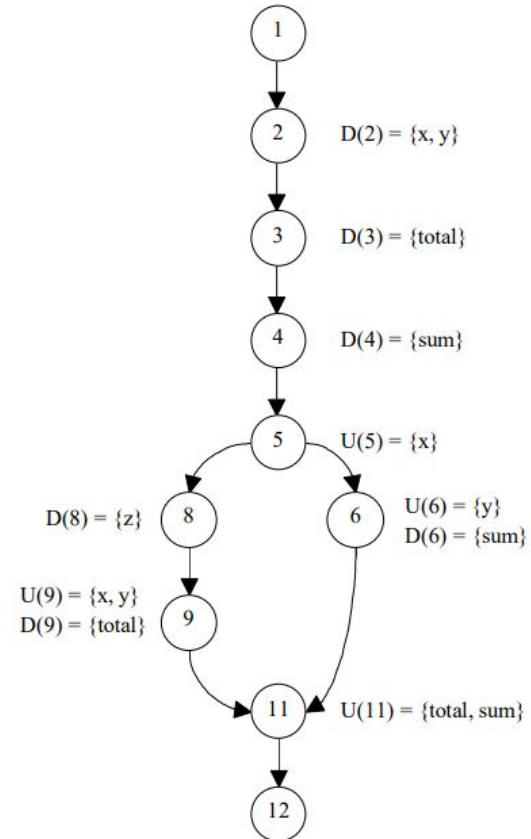
`z := derive some value from x;`

Cover all paths from a variable definition (write) to any uses (reads) of that variable *before* its value is overwritten = a new definition

Data flow is about reads and writes, not variable declarations

Example

```
1. begin
2.   read ( x, y );
3.   total := 0.0;
4.   sum := 0.0;
5.   if x <= 1
6.     then sum := y
7.     else begin
8.       read (z);
9.       total := x * y;
10.    end;
11.   write (total, sum);
12. end.
```

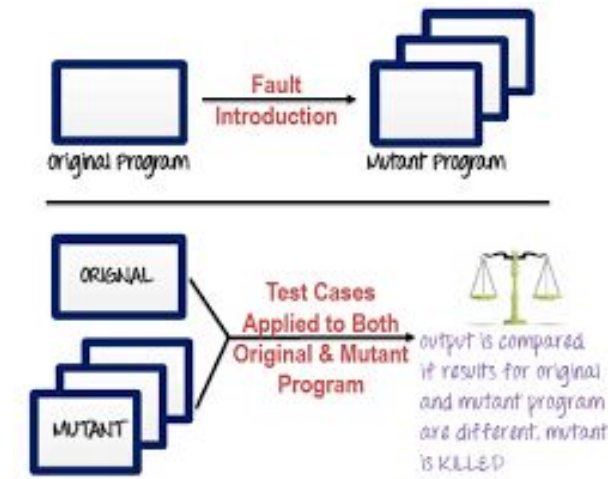


Mutation Testing

Intuition: Does your test suite actually detect bugs? Check by changing your code to intentionally insert a bug, then run your test suite. Did any test that passed with the original code and exercises the changed code now fail with the modified code? If not, then your test suite is not good enough to detect the bug you inserted. Repeat with another bug, etc.

Example mutation testing tools:

[PIT](#) for Java, [Mull](#) for C and C++,
[Stryker](#) for JS



Mutation Testing Terminology

Mutation Analysis = another name for Mutation Testing

Mutant = small change in one method, typically trying to insert a bug; otherwise same codebase and same tests

Mutation Score = number of mutants killed / total number of non-equivalent mutants

Equivalent Mutants = functionally equivalent to the original program, or to some other mutant generated using a different mutation operator and/or mutating a different method; remove all equivalent mutants before testing with the rest of the mutants to compute mutation score



Mutation Operators

Statement deletion

Statement duplication or insertion, e.g., goto fail

Replacement of boolean subexpressions with *true* and *false*

Replacement of some arithmetic operations with others, e.g., + with *, - with /

Replacement of some boolean relations with others, e.g., > with >=, == and <=

Replacement of variables with others from the same scope (variable types must be compatible)

Changes in synchronization concerned with concurrency and multi-threading

Insert Security Vulnerability (from [CVEs](#) and [CWEs](#))



Original Method	With Embedded Mutants
<pre>int Min (int A, int B) { int minVal; minVal = A; if (B < A) { minVal = B; } return (minVal); } // end Min</pre>	<pre>int Min (int A, int B) { int minVal; minVal = A; minVal = B; if (B < A) if (B > A) if (B < minVal) { minVal = B; Bomb(); minVal = A; minVal = failOnZero (B); } return (minVal); } // end Min</pre>

Mutation Testing Relies On...

- “Competent Programmer Hypothesis”: Most software faults introduced by experienced programmers are due to small errors
- “Coupling Effect Hypothesis”: Simple faults can cascade to form emergent faults
- “RIP Model”:
 1. A test must *reach* the mutated statement
 2. Test input data should *infect* the program state by causing different program states for the mutant vs. the original program
 3. The incorrect program state must *propagate* to the program's (or method's) output so it can be checked by the test



Remaining Assignments

→ [Second Iteration Demo](#) due this Thursday, December 7, by 11:59pm

Second Assessment will be available this Friday December 8 at 12:01am and due Monday December 11 at 11:59pm (4 days minus two minutes)

[Demo Day](#) is Monday, December 18, 10am-4pm, signup sheet is [here](#)



How to Answer
**Why Do We
Have to Do
This?**

Thursday

finish whatever we didn't get to today
test oracles and pseudo-oracles



Thursday.

The most useless day, it
only exists as a reminder
that it's been a really long
week, and it's still not over.

© sayingsimages.com



Ask Me Anything